Лабораторная работа №2

Включение и агрегирование компонентов ACOM

Компоненты EcoIntegralSimpson и Eco.Lab2

Содержание

1. Постановка задачи 2
2. Схема взаимодействия компонентов 3
3. Публичные интерфейсы 4
4. QueryInterface и управление временем жизни 5
5. Реализация компонентов 6
6. Тестирование 7
7. Результаты и анализ 8
8. Выводы и воспроизведение 9
9. Использование ИИ в работе 10

Выполнил: Михаил Сибиряков

Группа: 24ТИДИ

2026

1. Постановка задачи

Цель лабораторной работы — изучить включение и агрегирование компонентов в модели ACOM. Для этого компонент EcoIntegralSimpson из первой работы расширен арифметическими операциями с использованием учебных калькуляторов A–E. Также разработан внешний компонент Eco.Lab2, который агрегирует интегратор и предоставляет собственные калькуляторные интерфейсы.

В работе решаются следующие задачи:

- добавить сложение, вычитание, умножение и деление через готовые компоненты;
- продемонстрировать включение и агрегирование всех калькуляторов A–E;
- реализовать поддержку внешнего агрегирования EcoIntegralSimpson;
- продемонстрировать свойства интерфейсов и переходы через QueryInterface;
- сохранить численные методы и результаты первой лабораторной работы.

Интегратор по-прежнему использует адаптивный метод Симпсона и поддерживает три вещественных типа. Калькуляторы выполняют отдельные целочисленные операции. Их добавление расширяет интерфейсы компонента, не меняя алгоритм интегрирования.

Учебные калькуляторы A–E

В работе используются пять готовых калькуляторов из учебных материалов InsideACOM. Сложение и вычитание предоставляются через интерфейс `IEcoCalculatorX`, обозначаемый далее X. Для умножения и деления используется `IEcoCalculatorY`, обозначаемый Y.

Калькулятор A самостоятельно реализует X. Калькулятор B выполняет те же операции и дополнительно поддерживает агрегирование внешним объектом. Калькулятор C самостоятельно реализует оба интерфейса, X и Y.

Калькуляторы D и E показывают повторное использование готовой реализации. Оба самостоятельно реализуют Y. Для получения X компонент D включает B и при его недоступности пробует A, а компонент E агрегирует B.

При включении внешний компонент хранит интерфейс внутреннего объекта и перенаправляет ему вызовы. При агрегировании клиент получает интерфейс внутреннего объекта, а его методы Unknown обращаются к управляющему Unknown внешнего компонента. В работе включение и агрегирование сочетаются: EcoLab2 агрегирует интегратор, а интегратор включает калькуляторы A и D.

2. Схема взаимодействия компонентов

Каждый прямоугольник на схеме обозначает отдельный объект. Стрелка направлена от внешнего компонента к внутреннему и подписана способом их соединения. В блоках калькуляторов указаны операции, для которых они используются в этой связи.

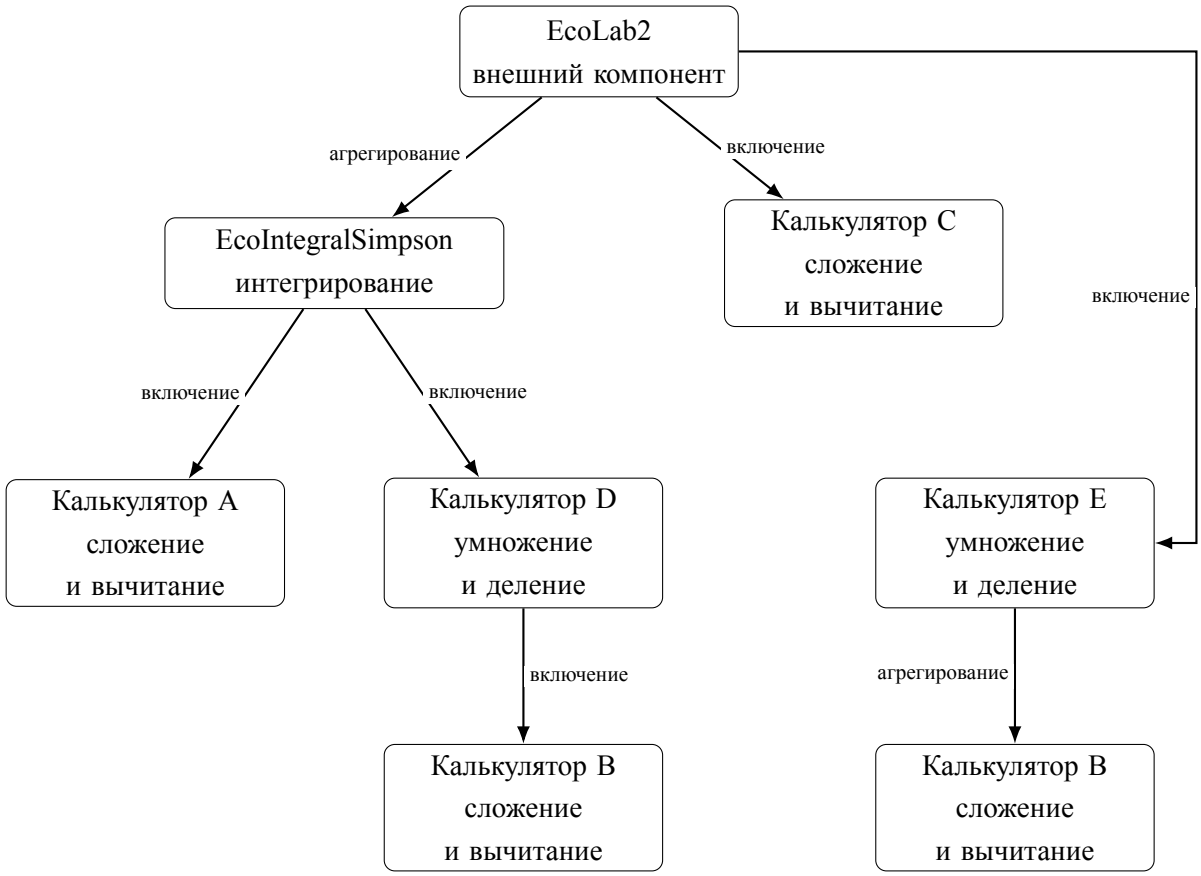


Схема показывает состав объектов. При выполнении одной операции используется соответствующая ей ветвь. Для клиента, создавшего EcoLab2, вызовы распределяются так:

- сложение и вычитание перенаправляются в калькулятор C;
- умножение и деление перенаправляются в калькулятор E;
- интегрирование выполняется объектом EcoIntegralSimpson через его интерфейс `IEcoAdvancedMath`.

При работе через EcoLab2 сложение и вычитание выполняет калькулятор C, а умножение и деление — E. Калькуляторы A и D используются для арифметических операций при самостоятельном создании интегратора.

Внизу показаны два отдельных экземпляра B. Они нужны для сложения и вычитания в самих D и E; эти операции проверяются также непосредственно через D/E. Умножение и деление D/E выполняют самостоятельно.

3. Публичные интерфейсы

Сохранённый интегратор

Интерфейс `IEcoAdvancedMath` содержит три варианта метода интегрирования:

- `integral` — для типа `double_t`;
- `integralf` — для типа `float_t`;
- `integrall` — для типа `long double`.

Callback возвращает число и принимает необязательный заимствованный контекст. Интегратор возвращает статус и записывает результат Q и оценку E только при успехе. При ошибке выходы сохраняются. Допуски конечны, неотрицательны, хотя бы один положителен; критерий завершения:

$$E \leq \max(\varepsilon_{abs}, \varepsilon_{rel}|Q|).$$

Сохранены адаптивное уточнение, предел 4096 подотрезков, обработка равных и обратных границ. Оценка E не является гарантированной границей ошибки для произвольной функции. Численные тела методов совпадают с базовой версией.

Арифметические операции EcoLab2

В EcoLab2 арифметическую операцию можно вызвать двумя способами. Через интерфейсы X/Y вызывается отдельный метод для нужного действия, который непосредственно возвращает вычисленное число.

Другой способ — общий метод Calculate интерфейса `IEcoLab2`. Он вызывает те же калькуляторы C/E, возвращает код состояния и записывает результат в переменную, адрес которой передал клиент. Параметр `me` — указатель на интерфейс EcoLab2, а `a` и `b` — числа, над которыми выполняется действие. Параметр `operation` выбирает операцию: 1 — сложение, 2 — вычитание, 3 — умножение, 4 — деление. В `result` передаётся адрес переменной, в которую нужно записать ответ.

Например, если `lab` — полученный интерфейс IEcoLab2, сложение $3 + 5$ вызывается следующим образом:

```
int32_t value = 0;
int16_t status = 0;
status = lab->pVTbl->Calculate(lab, 1, 3, 5, &value);
```

После успешного вызова `status` равен нулю, а `value` содержит 8. Код состояния сообщает об успехе или ошибке отдельно от результата вычисления.

Перед вычислением Calculate проверяет номер операции и переданные указатели. Для деления запрещён нулевой делитель; перед вычитанием и делением также проверяется, что ответ помещается в тип результата учебного калькулятора. При ошибке метод возвращает код ошибки и сохраняет прежнее значение результата. Деление остаётся целочисленным: например, $9/2$ даёт 4, дробная часть отбрасывается.

4. QueryInterface и управление временем жизни

Единый управляющий Unknown

Интерфейс `IEcoUnknown` содержит три служебных метода. QueryInterface позволяет получить другой интерфейс объекта, AddRef удерживает объект, а Release освобождает ссылку на него. Эти методы входят также в состав арифметических интерфейсов и интерфейса интегрирования.

При агрегировании публичные интерфейсы интегратора передают вызовы этих трёх методов внешнему компоненту EcoLab2. Поэтому запрос Unknown через любой из них возвращает один и тот же указатель на Unknown объекта EcoLab2. Этот интерфейс называется управляющим: он определяет доступные интерфейсы и управляет временем жизни всей композиции. Само интегрирование выполняется внутри EcoIntegralSimpson.

Неделегирующий Unknown интегратора

У интегратора есть отдельный неделегирующий Unknown. Его методы AddRef и Release изменяют счётчик ссылок самого интегратора и не передают эти вызовы в EcoLab2. Через его QueryInterface внешний компонент получает интерфейс интегрирования.

При создании агрегированного интегратора EcoLab2 передаёт свой управляющий Unknown и первоначально запрашивает только `IEcoUnknown`. Фабрика возвращает неделегирующий интерфейс, ссылку на который сохраняет EcoLab2. Публичный интерфейс интегрирования в поле EcoLab2 не сохраняется: он удерживал бы внешний объект и создал бы цикл ссылок между компонентами.

Если интегратор создан самостоятельно, управляющим для его публичных интерфейсов служит его собственный неделегирующий Unknown.

Пример переходов и освобождения объектов

1. Клиент запрашивает у EcoLab2 интерфейс интегрирования. EcoLab2 получает его через неделегирующий Unknown интегратора. Эта ссылка удерживает всю композицию.
2. Через интерфейс интегрирования клиент запрашивает X. Запрос обрабатывает EcoLab2 и возвращает интерфейс сложения и вычитания, использующий калькулятор C.
3. Клиент освобождает первоначальный интерфейс IEcoLab2. Пока у него остаётся интерфейс интегрирования, EcoLab2 и внутренние объекты продолжают работать.
4. После освобождения последнего публичного интерфейса EcoLab2 удаляется и освобождает интегратор через его неделегирующий Unknown, а также калькуляторы C/E. Интегратор освобождает A/D, а D и E — свои экземпляры B.

При ошибке создания фабрика интегратора очищает его через неделегирующий Unknown. Она не вызывает Release для EcoLab2, потому что не увеличивала его счётчик ссылок.

5. Реализация компонентов

Компоненты и C-клиенты написаны на C89 без вызовов libc. Для памяти используется `IEcoMemoryAllocator1`, для создания компонентов — интерфейсная шина Eco, для вывода — файловый сервис.

Основные файлы компонентов

Проекты EcoIntegralSimpson и Eco.Lab2 находятся в каталоге `Components`. В каждом проекте файлы реализации размещены в SourceFiles:

- `CEcoIntegralSimpson.c` — существующий файл первой работы. Добавлены включение A/D, арифметические интерфейсы и делегирующие методы Unknown. Реализован неделегирующий Unknown для внешнего агрегирования; численные методы Симпсона сохранены.
- `CEcoIntegralSimpsonFactory.c` — существующая фабрика интегратора. Добавлено создание с внешним управляющим Unknown и освобождение частично созданного объекта через его неделегирующий интерфейс при ошибках.
- `CEcoLab2.c` — новый файл реализации EcoLab2. В нём создаются включённые C/E и агрегированный интегратор, реализованы переходы между интерфейсами, метод Calculate и освобождение внутренних объектов.
- `CEcoLab2Factory.c` — новая фабрика EcoLab2. Она выделяет память, запускает инициализацию, выдаёт запрошенный интерфейс и очищает объект, если создание завершилось ошибкой.

У каждого компонента одна фабрика. Внутренние структуры описаны в HeaderFiles, публичные интерфейсы — в SharedFiles. Сохранена структура Eco Wizard.

Файлы тестового клиента

В корневом каталоге `UnitTestFiles/SourceFiles` находятся два C-файла:

- `EcoLab2.c` — пять сценариев второй лабораторной: арифметика интегратора и EcoLab2, использование B через D/E, переходы между интерфейсами и время жизни агрегированного объекта.
- `Lab2TestRegistration.c` — регистрация фабрик калькуляторов A–E в интерфейсной шине. Используется обоими тестовыми клиентами для создания зависимостей компонентов.

Клиент использует реальные компоненты и системные сервисы Eco. Прежний клиент EcoIntegralSimpson.c находится в `UnitTestFiles/SourceFiles` проекта интегратора. В него добавлена регистрация калькуляторов; прежние тесты интегрирования сохранены.

6. Тестирование

Клиент второй лабораторной содержит пять сценариев:

1. **Самостоятельный интегратор.** Через X и Y выполняются $3 + 5 = 8$, $3 - 5 = -2$, $3 \cdot 5 = 15$ и $-9/2 = -4$. Операции используют включённые A и D.
2. **EcoLab2.** Те же четыре операции выполняются через C/E. Отдельно вызываются четыре операции Calculate, проверяются неизвестный номер операции и деление на ноль. В обоих ошибочных случаях результат сохраняется. Через полученный у EcoLab2 интерфейс IEcoAdvancedMath вычисляется $\int_0^1 x^2\,dx = 1/3$.
3. **Использование B.** У D и E получается интерфейс Y, через него запрашивается X и выполняется $3+5 = 8$. Так показывается использование сложения из B через включение в D и агрегирование в E.
4. **Переходы между интерфейсами.** Для двух собственных компонентов перебираются все пары публичных интерфейсов, включая переход к тому же интерфейсу. Через каждый интерфейс дополнительно запрашивается Unknown и сравнивается с Unknown этого объекта.
5. **Время жизни агрегата.** У EcoLab2 получается IEcoAdvancedMath, после чего исходный IEcoLab2 освобождается. Через оставшийся интерфейс выполняется интегрирование и снова получается IEcoLab2. После освобождения этой дополнительной ссылки последний Release интерфейса интегрирования должен вернуть нулевое число ссылок.

Четыре публичных интерфейса интегратора дают $4 \cdot 4 = 16$ переходов, пять интерфейсов EcoLab2 — $5 \cdot 5 = 25$. Всего проверяется 41 пара:

Объект	Публичные интерфейсы	Пар
EcoIntegralSimpson	Unknown, AdvancedMath, X, Y	16
EcoLab2	Unknown, Lab2, AdvancedMath, X, Y	25
Итого		41

В каждом сценарии `Check` проверяет ожидаемый результат и выводит PASS или FAIL. На пять сценариев приходится соответственно 7, 15, 6, 61 и 5 проверок; ещё три проверяют регистрацию компонентов. Всего выполняется 97 проверок. В число 61 входят 41 переход, получение девяти исходных интерфейсов, девять сравнений Unknown и создание двух объектов.

Регрессионный клиент первой работы запускается отдельно. Он сохраняет 135 проверок интегратора для трёх числовых типов и результаты независимого сравнения с mpmath.

7. Результаты и анализ

Для проверки использована статическая сборка Linux x86_64 с GCC 13.2.0 из Eco toolchain. После чистой сборки запущены клиент второй лабораторной, регрессионные проверки интегратора и независимое сравнение с mpmath. Результаты приведены в таблице.

Проверка	Результат
Проверки клиента лабораторной 2	97 PASS, 0 FAIL
Направленные пары публичных интерфейсов	41
Регрессионный UnitTest интегратора	135 PASS, 0 FAIL
Тесты Python-компаратора	3 OK
Независимые сравнения с mpmath	30/30
Аудит собственных объектных файлов	Вызовов libc нет

Интегрирование сохранило исходные результаты независимого сравнения:

Тип	Сравнения	Максимальная фактическая абсолютная ошибка
float	10/10	$2.5431315 \cdot 10^{-6}$
double	10/10	$9.9972799 \cdot 10^{-12}$
long double	10/10	$1.9838356 \cdot 10^{-13}$

Эти значения относятся к фиксированному набору регрессионных примеров первой работы. Эталон — mpmath 1.3.0, tanh–sinh при 50 десятичных знаках с проверкой устойчивости при 70; результаты сверены с аналитическими первообразными.

Сравнение включения и агрегирования

При включении наш компонент сам реализует X и Y, а их методы вызывают соответствующие методы включённых калькуляторов. Для каждой арифметической операции нужен метод перенаправления в нашем компоненте.

При агрегировании EcoLab2 выдаёт клиенту интерфейс IEcoAdvancedMath внутреннего интегратора. Методы интегрирования в EcoLab2 повторно реализовывать не требуется. При этом запросы интерфейсов и учёт ссылок должны проходить через общий управляющий Unknown компонента EcoLab2, чтобы клиент работал с агрегатом как с одним объектом.

8. Выводы и воспроизведение

Компонент EcoIntegralSimpson расширен калькуляторными интерфейсами и поддержкой агрегирования. Создан EcoLab2, который предоставляет собственные арифметические интерфейсы и использует интерфейс интегрирования внутреннего компонента. На этой схеме продемонстрированы включение и агрегирование с использованием всех учебных калькуляторов A–E.

В UnitTest успешно выполнены 97 проверок. Проверены все переходы между публичными интерфейсами двух собственных компонентов. Через каждый интерфейс получается общий Unknown своего объекта. Сценарий времени жизни показал, что интерфейс интегрирования остаётся работоспособным после освобождения исходного IEcoLab2. После доработки компонента успешно выполнены также все 135 проверок интегратора и 30 сравнений с mpmath.

Сборка и запуск

Для сборки и полной проверки проекта в Linux x86_64 из корневого каталога выполняются команды:

```
cd AssemblyFiles/Linux/gcc_v132
./build.sh verify
```

Команда `verify` собирает компоненты и оба консольных клиента, запускает проверки второй лабораторной и прежние проверки интегратора. Затем выполняются сравнение результатов интегрирования с mpmath и проверка отсутствия вызовов libc в собственных объектных файлах.

Чтобы увидеть подробный вывод пяти сценариев второй лабораторной, из того же каталога запускается уже собранный клиент:

```
./build.sh lab2-run
```

В консоль выводятся проверяемые операции, переходы QueryInterface и общий итог проверки. Для сборки используются установленные Eco.Framework и Eco toolchain, для независимого сравнения результатов — Python 3 и mpmath.

9. Использование ИИ в работе

При выполнении лабораторной использовался ИИ-агент OpenAI Codex: модель GPT-6.1 Sol (`gpt-6.1-sol`), уровень reasoning — `xhigh`. Работа с агентом строилась итеративно. Сначала выполнялись поиск и анализ учебных материалов по включению и агрегированию компонентов. Затем согласовывались схема взаимодействия и интерфейсы компонентов. На основе этих решений генерировались прототипы кода компонентов и их тестов. Работоспособность проверялась сборкой, запуском UnitTest и сравнением с mpmath; результаты проверок использовались для уточнения реализации.

Для явного хранения контекста использовались отдельные Markdown-файлы: `docs/lab2/TASK.md` с требованиями, `docs/lab2/DESIGN.md` с архитектурными решениями, `AGENTS.md` с инструкциями для агента. Эти документы позволяли продолжать работу с учётом уже согласованных решений и ограничений, сохраняя контекст между итерациями.